Stock-Streams: Dynamic Portfolio Dashboard — Product Requirements Document (PRD)

1) Summary

Stock-Streams is a real-time portfolio dashboard where a user can manage equity holdings and see up-to-date performance metrics. Users can add stocks with quantity, purchase price, and exchange code (NSE/BSE). The system fetches live CMP (Yahoo Finance), P/E ratio and latest earnings (Google Finance), calculates PV, gain/loss, and portfolio weights, and streams updates to the UI.

2) Goals & Non-Goals

Goals

- Let users build and edit a portfolio and view real-time performance.
- Pull CMP (live quotes) and fundamentals (P/E, latest earnings) from 3rd-party sources.
- Display a powerful, mobile-first, sortable/filterable table; group by sector with roll-ups.
- Provide periodic refresh and WebSocket push for low-latency updates.

3) Assumptions & Constraints

- Yahoo/Google Finance do not offer stable public APIs; we will use scraping/unofficial libraries behind the backend. Build a provider abstraction so we can swap vendors if blocked.
- Market coverage: India (NSE/BSE). Symbols map to Yahoo tickers: `NSE:<CODE> -> <CODE>.NS`, `BSE:<CODE> -> <YF_BSE_CODE>.BO` (lookup table maintained by us).
- Update cadence: default 15s polling; WebSocket pushes whenever backend cache updates.
- Timezone/market hours: IST; after hours, CMP may be stale.

4) Success Metrics

- **P95 page load** < 2.5s on 4G; **P95 table interaction** < 100ms.
- **Data freshness**: CMP cache age ≤ 5s during market hours (configurable); fundamentals ≤ 24h.
- **Error rate** for quote fetch < 1% over market hours; automatic retries succeed ≥ 99%.

- **UX**: Add/edit/delete holding in ≤ 3 clicks.

5) High-Level Architecture

- **Frontend (Next.js 14, App Router)**: TanStack Table + React Hook Form + Zod; Tailwind CSS; TanStack Query for data fetching/caching. WebSocket client (socket.io) for live updates; fallback to polling.
- **Backend (NestJS)**: Modules: Auth (optional for v1), Portfolio, Instruments, Integrations (Yahoo/Google), Quotes, Fundamentals, Sockets, RateLimiter/Throttler. **DB:** PostgreSQL (*database:* `stock_streams_db`, *schema:* `public`) via **TypeORM** (migrations; snake_case). **Cache:** Redis (API/data cache). **Jobs:** BullMQ (quotes poller, fundamentals refresher).
- **Integrations**: Provider pattern with interfaces `IQuoteProvider`, `IFundamentalsProvider` and implementations for Yahoo/Google; circuit breakers, backoff, and per-provider rate limits.

Ops & Runtime (confirmed)

- **DB:** PostgreSQL `stock_streams_db` (schema `public`).
- **ORM:** TypeORM (Nest `TypeOrmModule`), `synchronize: true` only for local/dev; **migrations required** for prod.
- **Rate Limiting:** `@nestjs/throttler` (100 req/min window; 60s block on exceed) — can be tuned per route.
- **Cache:** Redis (env `REDIS_URL`), used for quotes/fundamentals caches + request caching.
- **Jobs:** BullMQ queues: `quotes` (every 5s during market hours) and `fundamentals` (07:00 IST daily). Concurrency defaults: quotes=10, fundamentals=3. Backoff: exp 100ms→1s x5; DLQ streams errors.
- **WebSockets:** Socket.IO gateway emits `quotes:update` and `portfolio:agg`.
- **Containers:** Dockerfile + docker-compose (dev/prod). API exposes `:5090`; Redis as `redis://default:<pass>@redis:6379`.

BACKEND PRD

> **Mode:** No authentication. **Single, global portfolio** (no users/portfolios tables).
> **One lot per instrument** for v1.

B1. Modules & Responsibilities (NestJS)

- **PortfolioModule**: CRUD for holdings; portfolio aggregation.

- **InstrumentModule**: Symbol directory & mapping (NSE/BSE → Yahoo code); sector metadata.
- **MarketDataModule**: Quote service with cache; batch fetch; WebSocket emission.
- **FundamentalsModule**: P/E & latest earnings provider; daily cache + on-demand refresh.
- **IntegrationsModule**: Provider interfaces & drivers (Yahoo/Google); circuit breakers.
- **SocketsModule**: Socket.IO gateway for `quotes:update`, `portfolio:agg`.
- **RateLimiterModule**: Per-IP and per-user quotas; sliding window in Redis.
- **JobsModule**: BullMQ schedules for periodic refresh (*/15s quotes*, *daily fundamentals*).

B2. Data Model (PostgreSQL + TypeORM)

Database: `stock_streams_db`, **schema:** `public`. Naming: snake_case tables/columns. All monetary fields use `numeric(14,2)` unless stated.

instruments

- id `uuid` PK
- name text not null
- exchange enum: `NSE|BSE`
- code text not null (e.g., `TCS`)
- yahoo_symbol text not null (e.g., `TCS.NS` or `500112.BO`)
- sector text
- status text default 'active'
- created_at timestamptz default now(), updated_at timestamptz default now()
- **UQ** (`exchange`, `code`), **UQ** (`yahoo_symbol`)

holdings *(single global portfolio; one lot per instrument for v1)*

- id `uuid` PK
- instrument_id uuid FK -> instruments(id) ON DELETE RESTRICT
- purchase_price numeric(14,2) not null
- quantity numeric(14,4) not null
- notes text
- created_at timestamptz default now(), updated_at timestamptz default now()
- **UQ** (instrument_id)

quotes_cache

- instrument_id uuid PK FK -> instruments(id) ON DELETE CASCADE
- price numeric(14,2) not null, currency text default 'INR', as_of timestamptz not null, provider text
- updated_at timestamptz default now()

fundamentals_cache

- instrument_id uuid PK FK -> instruments(id) ON DELETE CASCADE
- pe numeric(10,2), latest_eps numeric(10,2), earnings_period text, as_of timestamptz not null, provider text
- updated_at timestamptz default now()

TypeORM Entities

- Use `@Entity({ name: 'table', schema: 'public' })`; add `@UpdateDateColumn()` on `updated_at` where applicable. Indexes on instruments(code, exchange) and holdings(instrument_id).

B3. Calculations (authoritative on server)

- `investment = purchase_price * quantity`
- `present_value = latest_cmp * quantity`
- `gain_loss_amount = present_value - investment`
- `gain_loss_pct = (gain_loss_amount / investment) * 100`
- `portfolio_pct = investment / Σ(investment_all) * 100`
- Rounding: monetary values to 2dp; percentages to 2dp; P/E to 1dp.

B4. REST API Contracts (v1)

Portfolio

- `GET /api/v1/portfolio`
  1. **Response** `{ totals: { investment, present_value, gain_loss_amount, gain_loss_pct }, items: HoldingRow[] }`
- `POST /api/v1/portfolio`
  Request `{ name, exchange, code, sector, purchase_price, quantity, notes? }`
  Server steps (persistence-first):
  1. Resolve/insert **instrument** (find by `(exchange, code)`; if not found, create + map `yahoo_symbol`).
  2. **Insert** a **holding** row in `public.holdings` via TypeORM repo/manager **transaction**.
  3. Compute derived fields (investment, provisional portfolio %) on server.
  4. **Enqueue** a quotes fetch job (BullMQ `quotes`) and emit an immediate `portfolio:agg` for optimistic totals.
  5. Return **201** with the newly persisted row + computed fields.
     Idempotency: If the same instrument is added again and single-lot policy is enabled, return **409** with guidance or consolidate if `?merge=true`.
- `PATCH /api/v1/portfolio/:id`
  Request: `{ purchase_price?, quantity?, notes? }` → returns recomputed row.

Request: `{ purchase_price?, quantity?, notes? }` → returns recomputed row.
- `DELETE /api/v1/portfolio/:id` → 204

Market

- `GET /api/v1/market/quotes?ids=<instrument_id[]|codes[]>`
  Returns latest CMPs from cache; optional `force=true` triggers refresh (rate-limited).

Fundamentals

- `GET /api/v1/fundamentals?ids=<instrument_id[]|codes[]>` → `{ pe, latest_eps, earnings_period }`

Instruments

- `GET /api/v1/instruments/search?q=` → autocomplete by name/code; returns `{ id, name, exchange, code, yahoo_symbol, sector }`

Errors

- JSON `{ error: { code, message, details? } }`; HTTP codes: 400, 401, 404, 409, 429, 500.

B5. WebSocket Events (Socket.IO)

- **`quotes:update`**
  Payload: `{ instrument_id, yahoo_symbol, price, as_of }`
- **`portfolio:agg`**
  Payload: `{ totals, changed_item_id? }` emitted when a quote changes or holdings mutate.

Client subscribes to both and recomputes table cells; server also emits sector group roll-ups (optional key `sectors: { [sector]: { investment, pv, gl_amt, gl_pct } }`).

B6. Integrations & Fetch Strategy

Providers

- `IQuoteProvider.getQuotes(yahooSymbols: string[]): Promise<Quote[]>`
- `IFundamentalsProvider.getFundamentals(yahooSymbols: string[]): Promise<Fundamental[]>`

Yahoo (quotes)

- Implementation A: unofficial npm lib (e.g., yahoo-finance2) with batch endpoints.
- Implementation B: HTML/API fallback scraping using Cheerio + Rapid proxy; parse last trade price.

Google (fundamentals)

- Scrape Google Finance summary page per symbol; parse P/E (TTM) and latest EPS/earnings period; normalize.

Caching & TTLs

- Quotes TTL 5s; fundamentals TTL 24h. Redis keys: `q:<symbol>`, `f:<symbol>`.

Batching

- When FE requests portfolio, backend aggregates unique symbols; fetch in batches of 10; coalesce concurrent requests using a promise-in-flight map.

Resilience

- Circuit breaker (opossum) per provider; exponential backoff (100ms → 1s × 5); fallback to last cache value; mark cell as stale.

Rate Limiting

- Global: 120 req/min per IP; Endpoint specific: `/market/*` 30 req/min per token/IP; `/force=true` 6 req/min.

B7. Security

- Keys only server-side; `.env` and Vault support.
- CORS restricted to app origin.
- Input validation via class-validator + Zod (shared types).
- Helmet, compression; request logging with PII scrub.

B8. Observability

- Logs: structured (pino) with `symbol`, `provider`, `latency_ms`, `status`.
- Metrics: Prometheus—`quote_fetch_latency`, `cache_hit_ratio`, `socket_clients`, `provider_errors_total`.
- Traces: OpenTelemetry (NestJS plugin) around provider calls.

B9. Jobs & Scheduling (BullMQ)

- **Queues:** `quotes`, `fundamentals`.

- **Quotes poller (every 5s, market hours only)**: a repeatable BullMQ job (cron `*/5 * * * 1-5` gated by market-hours check). Collect active instrument symbols, fetch in batches (≤10), update Redis+DB `quotes_cache`, then **emit** `quotes:update` and recompute totals (without re-querying DB) to emit `portfolio:agg`. Concurrency 10; jitter ±1s to avoid thundering herd.
- **Fundamentals refresher (07:00 IST daily)**: cron `0 7 * * 1-5` (weekdays) with ability to run ad-hoc on cache miss. Refreshes `fundamentals_cache`; TTL 24h.
- **Backoff & DLQ:** exponential backoff (100ms→1s ×5); failures routed to dead-letter queue with alerting hook.
- **Market calendar:** config list for holidays; time window 09:15–15:30 IST.

B10. Testing Strategy

- **Unit**: formulas, mappers, symbol resolution, providers (with HTML fixtures).
- **Integration**: API endpoints with prisma test db; provider stubs.
- **E2E**: add/edit/delete flow; live updates with a mocked quote pump.

B11. Deployment

- **Containers:** Provided `Dockerfile` (multi-stage) and `docker-compose.yml` / `docker-compose.dev.yml`.
- **Services:** `api` (Nest, port 5090) + `redis` (7-alpine). DB may run on host (dev) or managed service.
- **Env vars:**
  - `PORT=5090`
  - **Postgres:** `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME=stock_streams_db`, `DATABASE_URL` (optional).
  - **Redis:** `REDIS_URL`, `REDIS_PASSWORD` (if set).
  - **Providers:** `YF_KEY?`, `GF_PROXY_URL?` (if used), `JWT_SECRET` (if auth enabled).
- **TypeORM:** Use migrations for prod; keep `synchronize=true` **only** in dev images.

B12. Acceptance Criteria (BE)

- **Persistence on add:** `POST /api/v1/portfolio` inserts a holding row into `public.holdings` (single global portfolio) and returns **201**; DB write succeeds even if downstream quote fetch fails.
- **Uniqueness:** Adding an already-present instrument returns **409** (one lot per instrument in v1) unless `?merge=true` is implemented.
- `/portfolio` returns accurate computed values with rounding rules and sector grouping in ≤ 500ms (cold ≤ 1.5s).
- Quotes cache achieves ≥ 80% hit ratio during market hours; fundamentals cached ≤ 24h.

- WebSocket emits `quotes:update` within ≤ 1s of cache write and `portfolio:agg` for totals.
- Rate limits enforced (100 req/min default) per IP and configurable per route.
- All write paths validated via DTO + class-validator; numeric bounds enforced; SQL injection safe.